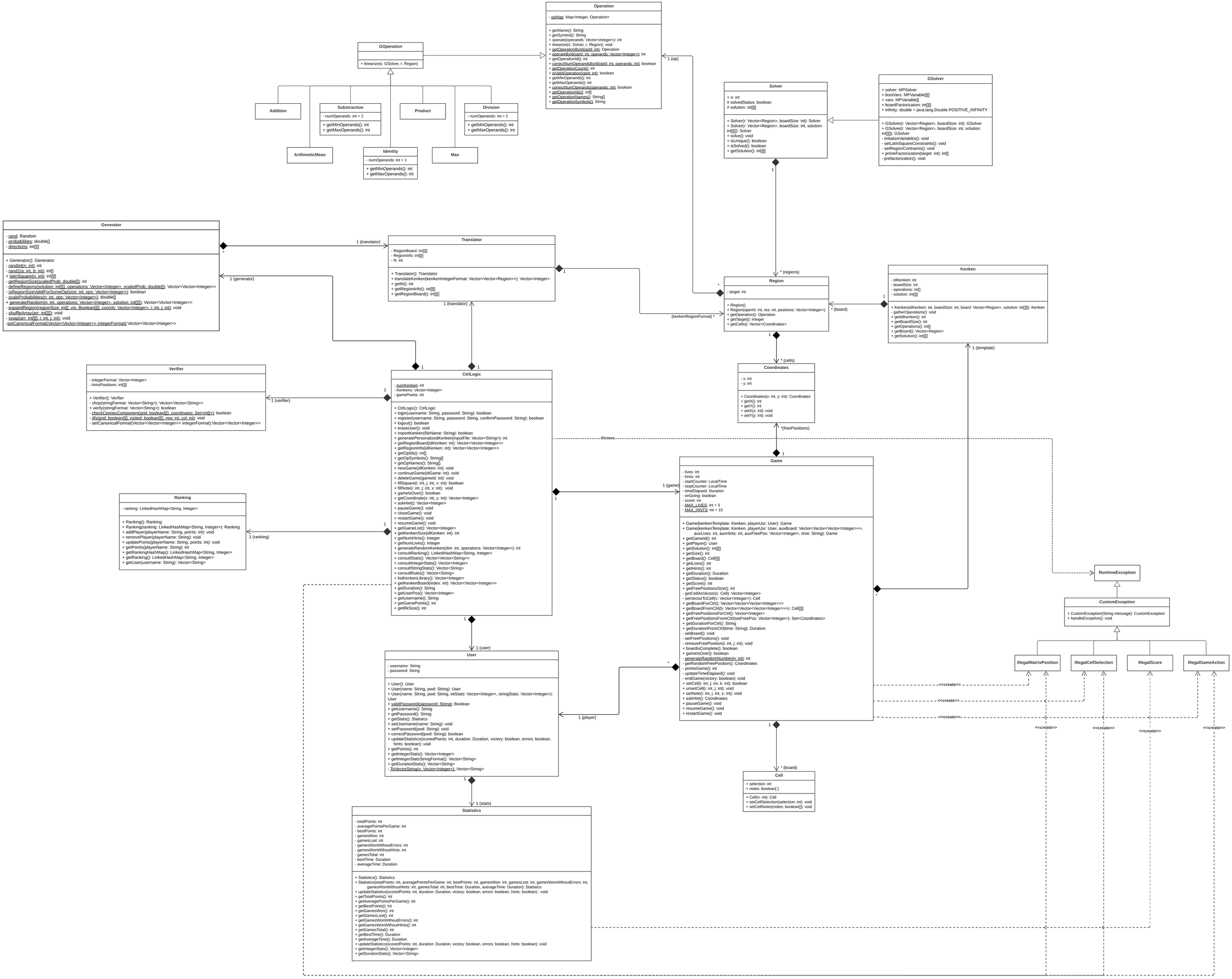
Operation
- opMap: Map<Integer, Operation>
+ getName(): String
+ getSymbol(): String
+ operate(operands: Vector<Integer>): int
+ linearize(s: Solver, r: Region): void
+ getOperationById(opId: int): Operation
+ operateById(opId: int, operands: Vector<Integer>): int
+ getOperationId(): int
+ correctNumOperandsById(opId: int, operands: int): boolean
+ getOperationCount(): int
+ isValidOperation(opId: int): boolean
+ getMinOperands(): int
+ getMaxOperands(): int
+ correctNumOperands(operands: int): boolean
+ getOperationIds(): int[]
+ getOperationNames(): String[]
+ getOperationSymbols(): String
GOperation
+ linearize(s: GSolver, r: Region)
Addition
Substraction
- numOperands: int = 2
+ getMinOperands(): int
+ getMaxOperands(): int
Product
Division
- numOperands: int = 2
+ getMinOperands(): int
+ getMaxOperands(): int
ArithmeticMean
Identity
- numOperands: int = 1
+ getMinOperands(): int
+ getMaxOperands(): int
Max
1 (op)
Solver
+ n: int
# solvedStatus: boolean
# solution: int[][]
+ Solver(r: Vector<Region>, boardSize: int): Solver
+ Solver(r: Vector<Region>, boardSize: int, solution: int[][]): Solver
+ solve(): void
+ isUnique(): boolean
+ isSolved(): boolean
+ getSolution(): int[][]
GSolver
+ solver: MPSolver
+ boolVars: MPVariable[][]
+ vars: MPVariable[]
+ boardFactorization: int[][]
+ infinity: double = java.lang.Double.POSITIVE_INFINITY
+ GSolver(r: Vector<Region>, boardSize: int): GSolver
+ GSolver(r: Vector<Region>, boardSize: int, solution: int[][]): GSolver
- initializeVariables(): void
- setLatinSquareConstraints(): void
- setRegionContraints(): void
+ primeFactorization(target: int): int[]
- prefactorization(): void
Generator
- rand: Random
- probabilities: double[]
- directions: int[][]
+ Generator(): Generator
- randint(n: int): int
- rand2(a: int, b: int): int[]
+ latinSquare(n: int): int[][]
- getRegionSize(scaledProb: double[]): int
- defineRegions(solution: int[][], operations: Vector<Integer>, scaledProb: double[]): Vector<Vector<Integer>>
- isRegionSizeValidForSomeOp(size: int, ops: Vector<Integer>): boolean
- scaleProbabilities(n: int, ops: Vector<Integer>): double[]
+ generateRandom(n: int, operations: Vector<Integer>, solution: int[][]): Vector<Vector<Integer>>
- expandRegion(regionSize: int[], vis: Boolean[][], coords: Vector<Integer>, i: int, j: int): void
- shuffleArray(arr: int[][]): void
- swap(arr: int[][], i: int, j: int): void
-setCanonicalFormat(Vector<Vector<Integer>> integerFormat):Vector<Vector<Integer>>
1 (translator)
1 (generator)
Translator
- RegionBoard: int[][]
- RegionInfo: int[][]
- N: int
+ Translator(): Translator
+ translateKenken(kenkenIntegerFormat: Vector<Vector<Region>>): Vector<Integer>
+ getN(): int
+ getRegionInfo(): int[][]
+ getRegionBoard(): int[][]
1 (translator)
(kenkenRegionFormat) *
* (regions)
Region
- target: int
+ Region()
+ Region(operId: int, res: int, positions: Vector<Integer>)
+ getOperation(): Operation
+ getTarget(): Integer
+ getCells(): Vector<Coordinates>
* (board)
Kenken
- idKenken: int
- boardSize: int
- operations: int[]
- solution: int[][]
+ Kenken(idKenken: int, boardSize: int, board: Vector<Region>, solution: int[][]): Kenken
- gatherOperations(): void
+ getIdKenken(): int
+ getBoardSize(): int
+ getOperations(): int[]
+ getBoard(): Vector<Region>
+ getSolution(): int[][]
1 (template)
* (cells)
Coordinates
- x: int
- y: int
+ Coordinates(x: int, y: int): Coordinates
+ getX(): int
+ getY(): int
+ setX(x: int): void
+ setY(y: int): void
*(freePositions)
Verifier
- integerFormat: Vector<Integer>
- hintsPositions: int[][]
+ Verifier(): Verifier
- chop(stringFormat: Vector<String>): Vector<Vector<String>>
+ verify(stringFormat: Vector<String>): boolean
- checkConnexComponent(grid: boolean[][], coordinates: Set<int[]>): boolean
- dfs(grid: boolean[][], visited: boolean[][], row: int, col: int): void
- setCanonicalFormat(Vector<Vector<Integer>> integerFormat):Vector<Vector<Integer>>
1 (verifier)
CtrlLogic
- numKenken: int
- Kenkens: Vector<Integer>
- gamePoints: int
+ CtrlLogic(): CtrlLogic
+ login(username: String, password: String): boolean
+ register(username: String, password: String, confirmPassword: String): boolean
+ logout(): boolean
+ eraseUser(): void
+ importKenken(fileName: String): boolean
+ generatePersonalizedKenken(inputFile: Vector<String>): int
+ getRegionBoard(idKenken: int): Vector<Vector<Integer>>
+ getRegionInfo(idKenken: int): Vector<Vector<Integer>>
+ getOpIds(): int[]
+ getOpSymbols(): String[]
+ getOpNames(): String[]
+ newGame(idKenken: int): void
+ continueGame(idGame: int): void
+ deleteGame(gameId: int): void
+ fillSquare(i: int, j: int, x: int): boolean
+ fillNote(i: int, j: int, x: int): void
+ gameIsOver(): boolean
+ getCoordinate(x: int, y: int): Vector<Integer>
+ askHint(): Vector<Integer>
+ pauseGame(): void
+ closeGame(): void
+ restartGame(): void
+ resumeGame(): void
+ getGameList(): Vector<Integer>
+ getKenkenSize(idKenken: int): int
+ getNumHints(): Integer
+ getNumLives(): Integer
+ generateRandomKenken(dim: int, operations: Vector<Integer>): int
+ consultRanking(): LinkedHashMap<String, Integer>
+ consultStats(): Vector<Vector<String>>
+ consultIntegerStats(): Vector<Integer>
+ consultStringStats(): Vector<String>
+ consultRules(): Vector<String>
+ listKenkenLibrary(): Vector<Integer>
+ getKenkenBoard(index: int): Vector<Vector<Integer>>
+ getDuration(): String
+ getUserPos(): Vector<Integer>
+ getUsername(): String
+ getGamePoints(): int
+ getRkSize(): int
throws
1 (game)
Game
- lives: int
- hints: int
- startCounter: LocalTime
- stopCounter: LocalTime
- timeElapsed: Duration
- onGoing: boolean
- score: int
- MAX_LIVES: int = 3
- MAX_HINTS: int = 10
+ Game(kenkenTemplate: Kenken, playerUsr: User): Game
+ Game(kenkenTemplate: Kenken, playerUsr: User, auxBoard: Vector<Vector<Vector<Integer>>>, auxLives: int, auxHints: int, auxFreePos: Vector<Integer>, time: String): Game
+ getGameId(): int
+ getPlayer(): User
+ getSolution(): int[][]
+ getSize(): int
+ getBoard(): Cell[][]
+ getLives(): int
+ getHints(): int
+ getDuration(): Duration
+ getStatus(): boolean
+ getScore(): int
+ getFreePositionsSize(): int
- getCellAsVector(c: Cell): Vector<Integer>
- setVectorToCell(v: Vector<Integer>): Cell
+ getBoardForCtrl(): Vector<Vector<Vector<Integer>>>
+ getBoardFromCtrl(b: Vector<Vector<Vector<Integer>>>): Cell[][]
+ getFreePositionsForCtrl(): Vector<Integer>
+ getFreePositionsFromCtrl(setFreePos: Vector<Integer>): Set<Coordinates>
+ getDurationForCtrl(): String
+ getDurationFromCtrl(time: String): Duration
- setBoard(): void
- setFreePositions(): void
- removeFreePosition(i: int, j: int): void
+ boardIsComplete(): boolean
+ gameIsOver(): boolean
- generateRandomNumber(n: int): int
- getRandomFreePosition(): Coordinates
- pointsGame(): int
- updateTimeElapsed(): void
- endGame(victory: boolean): void
+ setCell(i: int, j: int, k: int): boolean
+ unsetCell(i: int, j: int): void
+ setNote(i: int, j: int, x: int): void
+ askHint(): Coordinates
+ pauseGame(): void
+ resumeGame(): void
+ restartGame(): void
Ranking
- ranking: LinkedHashMap<String, Integer>
+ Ranking(): Ranking
+ Ranking(ranking: LinkedHashMap<String, Integer>): Ranking
+ addPlayer(playerName: String, points: int): void
+ removePlayer(playerName: String): void
+ updatePoints(playerName: String, points: int): void
+ getPoints(playerName: String): int
+ getRankingHashMap(): LinkedHashMap<String, Integer>
+ getRanking(): LinkedHashMap<String, Integer>
+ getUser(username: String): Vector<String>
1 (ranking)
1 (user)
User
- username: String
- password: String
+ User(): User
+ User(name: String, pwd: String): User
+ User(name: String, pwd: String, intStats: Vector<Integer>, stringStats: Vector<Integer>): User
+ validPassword(password: String): Boolean
+ getUsername(): String
+ getPassword(): String
+ getStats(): Statistics
+ setUsername(name: String): void
+ setPassword(pwd: String): void
+ correctPassword(pwd: String): boolean
+ updateStatistics(scoredPoints: int, duration: Duration, victory: boolean, errors: boolean, hints: boolean): void
+ getPoints(): int
+ getIntegerStats(): Vector<Integer>
+ getIntegerStatsStringFormat(): Vector<String>
+ getDurationStats(): Vector<String>
- ToVectorString(v: Vector<Integer>): Vector<String>
1 (player)
1 (stats)
Statistics
- totalPoints: int
- averagePointsPerGame: int
- bestPoints: int
- gamesWon: int
- gamesLost: int
- gamesWonWithoutErrors: int
- gamesWonWithoutHints: int
- gamesTotal: int
- bestTime: Duration
- averageTime: Duration
+ Statistics(): Statistics
+ Statistics(totalPoints: int, averagePointsPerGame: int, bestPoints: int, gamesWon: int, gamesLost: int, gamesWonWithoutErrors: int, gamesWonWithoutHints: int, gamesTotal: int, bestTime: Duration, averageTime: Duration): Statistics
+ updateStatistics(scoredPoints: int, duration: Duration, victory: boolean, errors: boolean, hints: boolean) : void
+ getTotalPoints(): int
+ getAveragePointsPerGame(): int
+ getBestPoints(): int
+ getGamesWon(): int
+ getGamesLost(): int
+ getGamesWonWithoutErrors(): int
+ getGamesWonWithoutHints(): int
+ getGamesTotal(): int
+ getBestTime(): Duration
+ getAverageTime(): Duration
+ updateStatistics(scoredPoints: int, duration: Duration, victory: boolean, errors: boolean, hints: boolean): void
+ getIntegerStats(): Vector<Integer>
+ getDurationStats(): Vector<String>
* (board)
Cell
+ selection: int
+ notes: boolean[.]
+ Cell(n: int): Cell
+ setCellSelection(selection: int): void
+ setCellNotes(notes: boolean[]): void
RuntimeException
CustomException
+ CustomException(String message): CustomException
+ handleException(): void
IllegalMatrixPosition
IllegalCellSelection
IllegalScore
IllegalGameAction
<<create>>